

MISAKA

耐量子 BlockDAG Layer-1、Consensus-Native EVM、 客観検証可能な Local/External LLM Work

The MISAKA Project

<https://github.com/MISAKA-BTC/misakas> · Explorer: <https://misakascan.com>

Whitepaper Version 2.0 – Research Specification – 2026-07-14

Abstract

要旨. MISAKA は *rusty-kaspa* を基盤とする PQ-only の耐量子 Layer-1 である。GHOSTDAG BlockDAG consensus と UTXO ledger を維持しつつ、classical attack に弱い consensus surface を ML-DSA-87 authorization、64-byte keyed BLAKE2b-512 consensus identity、additive lattice accumulator へ置き換える。さらに selected-parent EVM execution lane と、deep reorganization を gate し past-relative randomness を供給する DNS-style stake overlay を統合する。

Version 2 は PALW を追加する。PALW は deterministic LLM inference とその GEMM schedule を、上限付き block work component へ変換する audited-compute lane である。PALW は language model に parent、transaction、timestamp、difficulty、validity rule を選ばせない。高価な inference は block 生成前に実行され、provider receipt と trace commitment が固定され、全 leaf が公開され、future randomness で post-commit audit が選ばれ、maturity 後に one-shot work ticket となる。block producer は通常の deterministic template 内で active ticket を一度だけ消費する。full node は model を再実行せず、ticket、certificate、slot、fork bindings、lane difficulty、authorization、nullifier、compute cap を検証する。

本設計は nominal provider pair が同一 adversary に支配され得ると明示的に仮定する。replica agreement は consistency signal であり、2 回の独立実行の proof ではない。positive credit は minimum adversarial execution cost、future-randomized auditability、slashable exposure、reward escrow、proof-class subcap、permanent hash-work floor、deterministic GHOSTDAG dedup に基づく。Version 2 は通常の local model 利用も first-class PALW input とする。local application は利用者へ応答しながら private または replicated compute receipt を生成でき、第三者の有料依頼には signed demand receipt と escrow を要求する。token 数は直接 work にせず、SelfPrivate、SelfReplicated、ExternalReplicated に別々の objective multiplier、cap、fee、privacy obligation を適用する。本書は block-generation algorithm、objective verification predicate、DNS beacon、selected-parent state、attack model、staged activation を定義する。nonzero PALW work は testnet research target であり、現時点の mainnet activation を主張しない。

Version 2 の追加点

- LLM work-backed block lane を追加するが、block construction 自体は通常の deterministic node logic に限定する。
- objective verification を、selected-parent state と公開 commitment から全 validator が再現できる consensus predicate として定義する。committee certificate への盲目的信頼ではない。
- provider independence を安全仮定から外し、 $k = 2$ replica agreement を 2 実行分として数えない。
- pair assignment、audit selection、ticket slot に相互分離した 3 段階の future randomness を導入する。
- work/receipt/ticket nullifier、selected-parent-relative overlay、lane DAA、all-or-nothing compute cap、reward escrow を追加する。
- local LLM mining mode と paid external demand を追加するが、raw token count、requester identity、self-declared usefulness を consensus work にしない。
- validation、pruning、reorg、audit、adversarial test gate が完了するまで PALW は disabled または zero-credit とする。

目次

1	はじめに	2
1.1	本書の貢献	2
1.2	構成	2

2	Status, Scope, Claim Boundary	2
2.1	Normative vocabulary	2
2.2	Implementation status	3
2.3	Claim boundary	3
3	予備知識と記法	3
4	System Overview	4
4.1	Design goal	4
4.2	Non-goal	4
5	耐量子 Cryptographic Core	4
5.1	Transaction authorization	4
5.2	Hash64 identity	5
5.3	UTXO accumulator	5
5.4	Domain isolation	5
6	GHOSTDAG Consensus と Permanent Hash Lane	5
6.1	Propagation と version safety	5
7	DNS Finality と Consensus Randomness	5
7.1	二資源 deep-reorg gate	5
7.2	Past-relative bond snapshot	5
7.3	Beacon V2	5
7.4	Overlay commitment root	6
8	Selected-Parent EVM と Native-Asset Bridge	6
9	PALW Overview と Threat Model	6
9.1	Adversary capability	6
9.2	Security assumption	6
9.3	Provider independence を仮定しない	7
10	Deterministic LLM Work Object	7
10.1	Runtime profile	7
10.2	Job precommit と work unit	7
10.3	Operation schedule と trace	7
10.4	Output commitment と privacy	7
10.5	Nullifier hierarchy	8
11	Local LLM Participation と Paid Demand	8
11.1	Job class と privacy mode	8
11.2	Interactive local workflow	9
11.3	Raw token ではなく canonical compute unit	9
11.4	Demand receipt、escrow、wash-trade resistance	10
11.5	Credit schedule、class subcap、quantumization	10
11.6	Fixed block reward と role allocation	11
12	Provider Assignment と Sybil-Resilient Exposure	11
12.1	Past-relative provider snapshot	11
12.2	Liability-weighted selection	11
12.3	Control domain	11
12.4	Pair reuse、probation、exposure	11
13	三段階に分離した Randomness	12
13.1	Receipt commit/reveal	12
13.2	Audit 前の全 leaf 公開	12
14	Objective Audit, Dispute, Certification	12
14.1	Assurance class	13

14.2	Future audit draw	13
14.3	Canary と real-job audit	13
14.4	Mismatch resolution	13
14.5	Certificate semantics	14
15	Ticket Creation と LLM-Work-Backed Block Generation	14
15.1	Batch state machine	14
15.2	Target slot	14
15.3	Checkpoint chain commitment	14
15.4	Expected lane difficulty と eligibility	14
15.5	Header V3 field	15
15.6	Template algorithm	15
15.7	Authorization	15
16	Fast Consensus Verification	15
16.1	Canonical validation clause	15
16.2	GHOSTDAG ticket dedup	16
16.3	Pruning と IBD	16
17	Work Accounting、Compute Cap、Lane DAA	17
17.1	Minimum adversarial cost	17
17.2	Component recurrence	17
17.3	Effective work と attack bound	17
17.4	Lane-specific DAA	17
18	Reward、Slashing、Liveness	17
18.1	Reward escrow と settlement	17
18.2	Fraud expected return	18
18.3	Objective evidence	18
18.4	Historic work	18
18.5	Degraded/Halted mode	18
19	Security Analysis	19
19.1	Provider collusion	19
19.2	Fake/shortened computation	19
19.3	Ticket inventory/grinding	20
19.4	DNS/auditor compromise	20
19.5	Self-generated workload と token inflation	20
19.6	Circular external demand	20
19.7	Private-local downgrade attack	20
19.8	Prompt injection と generated content	20
19.9	Denial of service	20
19.10	Privacy と auditability	20
20	Rollout、Test Gate、Implementation Discipline	20
20.1	Activation stage	21
20.2	No-go condition	21
20.3	Mandatory test	21
21	関連研究	22
22	結論	22
A	Consensus Object Sketch	22
B	Objective Verifier Pseudocode	23
C	Parameter Classification	24

1. はじめに

MISAKA を動機づける圧力は三つある。第一は暗号である。classical elliptic-curve authorization と discrete-log 型 accumulator は十分な quantum adversary に対して安全性を維持できず、短い consensus hash は Grover search により margin を失う。第二は構造である。linear chain は propagation delay を orphan pressure に変えるが、BlockDAG は concurrent honest block を取り込み、GHOSTDAG によって順序づけられる [1]。第三は計算資源である。現代の network は有用な inference に大量の accelerator capacity を消費するが、これを単純に “proof of work” と呼ぶと破綻する。model execution は再実行が高価で、private data を含み、hardware-sensitive であり、共謀した provider が偽装しやすいからである。

Version 1 は最初の二つと EVM lane を扱った。Version 2 は第三の圧力に対して保守的な設計を追加する。*work creation* と *block acceptance* を分離し、provider は block が存在する前に deterministic inference profile を実行する。receipt、trace commitment、public leaf を future audit draw より先に固定する。objective check、maturity delay、economic exposure を通過した leaf だけが ticket となる。miner は一つの DAA interval と一つの fork-bound checkpoint に束縛された ticket を使う。block acceptance は hash、signature、membership proof、state lookup、integer comparison の bounded sequence に保たれる。

この分離は本質的である。language model は consensus oracle ではない。生成 text が transaction validity、selected parent、fork choice を決めてはならない。model output は service result であり、その canonical execution が bounded resource claim を構成する。consensus は claim の protocol history と one-shot use を検証する。

1.1. 本書の貢献

1. PQ UTXO ledger、consensus-native selected-parent EVM、PALW audited-compute work lane を一つの GHOSTDAG 上で定義する。
2. candidate block、selected-parent-relative consensus view、public/committed evidence だけから再現できる objective PALW verifier を定義する。IP、operator reputation、subjective model quality は入力にしない。
3. provider A/B が同一 adversary に支配される前提で安全性を評価する。replica matching は fault detection と double liability のために残すが、2 回の独立実行として credit しない。
4. job、provider assignment、audit selection、certificate、slot、lane difficulty、checkpoint commitment、authorization、ticket consumption を、separated future randomness で段階的に束縛する。
5. 日常の local LLM 利用を SelfPrivate/SelfReplicated receipt に変換し、paid external request には base compute の最大 1/5 かつ burn-backed の demand bonus のみを認める。
6. conservative component-work accounting を導入する。

$$E = H + \min(C_{\text{strong}} + \min(C_{\text{replica}}, r_{\text{replica}} H), 4H). \quad (1)$$

permanent hash floor H を残し、Replica class により厳しい subcap を課す。

7. activation、rollback、pruning、reward、slashing、privacy、adversarial test を仕様化し、struct が存在するだけで live protocol と見なさない。

1.2. 構成

Sections 2–8 で MISAKA base architecture を要約する。Sections 9–16 で LLM work、ticket、block construction、objective verification を定義する。Sections 17–19 で accounting、economics、attack を分析する。Section 20 で implementation と activation gate を示す。Appendix には object schema、verifier pseudocode、parameter classification を置く。

2. Status, Scope, Claim Boundary

2.1. Normative vocabulary

MUST、MUST NOT、SHOULD、MAY は target consensus rule を表す。*provisional* は testnet または engineering proposal であり mainnet promise ではない。*research* は current zero-credit stage の dependency ではない。

2.2. Implementation status

表 1. Version 2 の status boundary。

Component	Version 2 の記述	Publication 時点
PQ UTXO consensus	ML-DSA-87 authorization、Hash64 identity、lattice accumulator	Base design / implementation track
GHOSTDAG と hash lane	Permanent consensus floor と selected-parent ordering	Base design
DNS finality overlay	Reorg gate と past-relative state commitment	Base design、V2 beacon hardening 必須
Selected-parent EVM	Delayed mergeset acceptance を持つ consensus-native execution	Base design / staged implementation
PALW wire objects	Header-v3 fields、leaf、certificate、beacon、nullifier	Reserved / partial implementation
PALW nonzero work	Positive GHOSTDAG compute credit	未 activation
Objective real-job audit	Future-randomized rerun/dispute	nonzero credit 前に必須
Strong proof class	Succinct/transparent inference proof	Future research

本書は target hard-fork design である。current network が algo-4 block を受理する、または全 validation clause が production path に接続済みであるという claim ではない。finite activation fence を設定する前に、acceptance predicate、state rollback、template、RPC serialization、pruning witness、adversarial test が一致しなければならない。

2.3. Claim boundary

MISAKA の post-quantum claim は Section 5 の UTXO consensus authorization と consensus identity を対象とする。transport session、EVM account、external dependency、optional TEE の全てが PQ であるとは主張しない。

PALW の objective-verification claim も限定的である。full node は private network telemetry や provider の unsupported assertion を信頼せず PALW protocol satisfaction を判定できる。ただし Replica certificate は 2 台の物理 GPU が完全 model を実行した mathematical proof ではない。Replica class の安全性は probabilistic/economic であり、future-randomized audit、public commitment、slashable exposure、conservative credit、cap によって fake work を高コストかつ bounded にする。future StrongProof class は別 assurance model として別会計する。

3. 予備知識と記法

byte-string concatenation を $\parallel$ と書く。hash domain は length-delimited かつ network-bound である。 $\text{Hash64}_k(m)$ は domain key k の keyed BLAKE2b-512、 $\text{MerkleRoot}(S)$ は canonical Merkle root を表す。consensus integer は fixed width/fixed encoding であり、overflow は明示的 rule がない限り invalid である。

BlockDAG は block を vertex とする DAG $G = (V, E)$ であり、edge は block から parent へ向かう。GHOSTDAG は well-connected blue cluster を選び、各 block に selected parent $\text{sp}(B)$ 、blue score、cumulative blue work を与える [1]。selected chain は past に total order を誘導する。mergeset は selected parent と、その step で merge される eligible anticone block を含む。

PALW では次を区別する。

Work unit. Frozen profile と operation schedule の下の canonical deterministic inference quantum。

Candidate leaf. 一つの work unit と provider claim への public commitment。まだ block work ではない。

Active ticket. protocol、audit、certificate、maturity を通過した candidate leaf。

Source block. ticket を消費する block。GHOSTDAG consensus order で一度だけ credit される。

Past-relative view. node の current virtual tip や mempool ではなく candidate block の selected parent 時点の state。

Objective predicate. consensus data と cryptographic evidence から全 validator が再現できる deterministic predicate.

4. System Overview

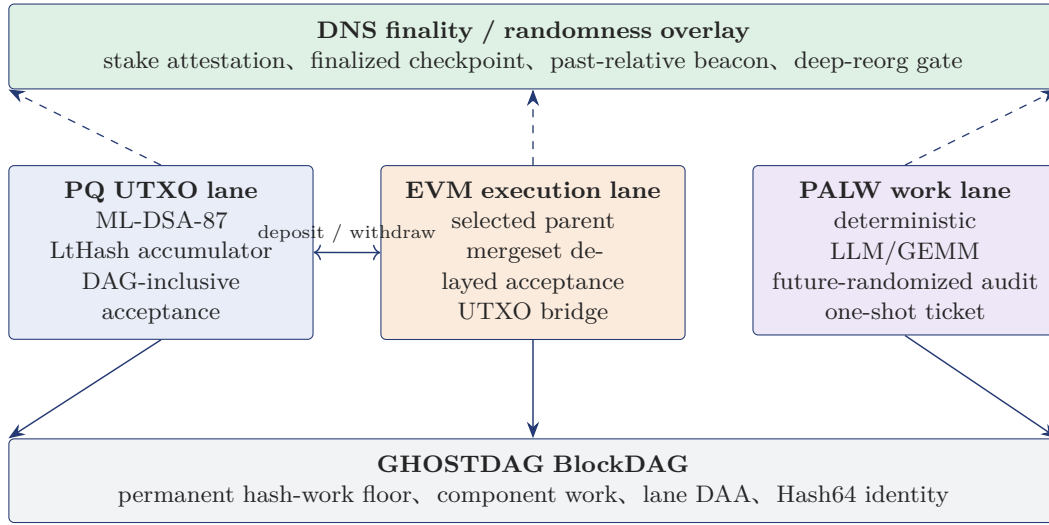


図 1. MISAKA Version 2 architecture. UTXO/EVM lane は state transition を実行し、PALW は pre-certified work component を bounded に追加する。DNS overlay は finalized anchor、stake-gated reorg protection、consensus randomness を供給する。

三つの lane は block identity、selected-parent state、propagation rule、GHOSTDAG order を共有するが、任意の trust assumption は共有しない。UTXO lane は PQ-only である。EVM lane は future PQ account format が導入されるまで独立の secp256k1 account domain を持つ。PALW は stateful smart contract を実行せず、provider に block content の変更権限を与えない。PALW output は ticket reference と bounded work increment である。

4.1. Design goal

1. Permanent かつ independently verifiable な hash-work floor を維持する。
2. ordinary block validation を fast/bounded に保ち、高価な inference を acceptance hot path へ入れない。
3. validity-relevant PALW input を全て past-relative にし、reorg、restart、pruning import 後も再現可能にする。
4. hidden-leaf、ticket stockpile、fork reuse、first-seen-order attack を防ぐ。
5. Sybil identity を solved identity theorem ではなく economic exposure problem として扱う [2]。
6. model usefulness、service quality、consensus work を分離する。意味的に低品質でも canonical execution なら work になり得るが、意味的に高品質でも unverifiable private job は consensus credit 0 である。
7. 全 prompt を公開せず local application が参加できるようにする一方、strict non-disclosure は work 0、positive private credit は committed future-audit path 必須とする。
8. external-demand uplift を circular にしない。requester signature 単体では不足し、bonus は irreversible fee burn と selected-parent-relative epoch cap の両方で bound する。

4.2. Non-goal

本設計は、二つの provider key が異なる人、法人、rack、GPU であることを証明しない。arbitrary private inference に audit-capable path または strong proof なしで positive credit を与えない。TEE attestation を consensus root にしない。第二 wallet が request へ署名しただけで external-demand credit を与えない。fraud が後日発見されても historic GHOSTDAG work を遡及削除しない。LLM output の factual correctness を canonical execution と同一視しない。

5. 耐量子 Cryptographic Core

5.1. Transaction authorization

PQ UTXO lane は FIPS 204 の ML-DSA-87 を使う [3]。consensus/wallet encoding は network、transaction version、input index、script context、sighash mode を commit する。legacy ECDSA、Schnorr、P2SH

authorization path は PQ consensus mode で表現不能にする。optional transport confidentiality は FIPS 203 の hybrid ML-KEM を利用できるが、transport security と consensus authorization は別である [4]。

5.2. Hash64 identity

block hash、transaction id、Merkle root、UTXO commitment、parent reference、PALW object id、overlay root は 64-byte domain-separated BLAKE2b-512 を使用する [5]。keyed mode は secrecy ではなく domain separation のためであり、length prefix と version tag が object family の ambiguous preimage を防ぐ。

5.3. UTXO accumulator

UTXO set は LtHash engineering line に沿う additive lattice accumulator で commit する [6]。add/delete は incremental であり、64-byte header commitment が complete set を束縛する。algebra、serialization、overflow behavior は consensus rule である。

5.4. Domain isolation

EVM account signature、PALW provider signature、DNS bond signature、ticket authorization、UTXO transaction signature は別 domain とする。同じ PQ signature algorithm を使っても、provider receipt を block authorization として replay できない。

6. GHOSTDAG Consensus と Permanent Hash Lane

MISAKA は Kaspas から GHOSTDAG ordering と blue-work tip selection を継承する [1]。PALW activation 後も base hash lane は security floor として残る。header は Layer-1 algorithm id を持ち、consensus-critical Layer-0 finalizer が self-delimiting preimage 全体を 512-bit value にする。

live hash lane の conceptual construction は次である。

$$L_1 = \text{BLAKE2b512}(x) \parallel \text{SHA3-512}(x), \quad (2)$$

$$p_{512} = \text{BLAKE2b-512}_{k_{\text{pow}}}(\text{network} \parallel \text{algo} \parallel x \parallel \text{len}(L_1) \parallel L_1), \quad (3)$$

$$\text{accept} \iff \text{Uint512}(p_{512}) \leq \text{target}_{512}(\text{bits}). \quad (4)$$

exact header preimage、target conversion、work calculation は version で凍結する。PALW はこの check を弱めず、独自 validation/accounting を持つ second source class を追加する。

6.1. Propagation と version safety

unknown algorithm id と malformed version/field combination は panic せず fail closed にする。pre-v3 header は nonzero PALW field を運べず、post-v3 field は frozen order で block-hash preimage に入る。template と validator は同じ resolver を共有し、activation の二重実装を作らない。

7. DNS Finality と Consensus Randomness

7.1. 二資源 deep-reorg gate

DNS overlay は block production と分離される。active bonded validator が finalized anchor を attest する。stake-confirmed anchor を外れる candidate history は work rule と overlay stake rule の双方を満たす必要がある。ordinary short-range consensus を BFT に変えるのではなく、deep-reorg boundary に第二資源を導入する。

7.2. Past-relative bond snapshot

PALW epoch E ごとに eligible DNS set を commit window 前の cutoff で固定する。

$$\mathcal{D}_E = \{(o_i, pk_i, s_i, a_i) \mid \text{bond } i \text{ is active at cutoff}(E)\}. \quad (5)$$

o_i は bond outpoint、 pk_i owner key、 s_i effective stake、 a_i activation metadata である。quorum denominator は total eligible stake $S_E = \sum_{i \in \mathcal{D}_E} s_i$ であり、commit した stake だけではない。後の unbond/slash は historic denominator を書き換えないが、objective evidence は current funds に適用できる。

7.3. Beacon V2

各 participant は secret opening に commit する。

$$c_i = \text{Hash64}_{k_c}(\text{network} \parallel \text{fork} \parallel E \parallel o_i \parallel r_i), \quad (6)$$

$$q_i = \text{Hash64}_{k_r}(\text{network} \parallel \text{fork} \parallel E \parallel o_i \parallel r_i). \quad (7)$$

reveal phase 後、seed は already-public commitment ではなく actual opening から導く。

$$R_E = \text{Hash64}_{k_b}(R_{E-1} \parallel A_E \parallel \text{MerkleRoot}(\{q_i\}_{\text{valid}}) \parallel \text{MerkleRoot}(\{c_j\}_{\text{missing}}) \parallel E). \quad (8)$$

A_E は finalized DNS checkpoint と eligible-set root を束縛する。missing root は withholding を accountable にするが、last-revealer bias を自動的に消さない。reveal window、third-party retransmission、degraded mode、unhealthy beacon 時の PALW multiplier 0 を併用する [7, 8]。

7.4. Overlay commitment root

candidate block は selected parent 時点の overlay を解決する。Version 2 は mutable global snapshot ではなく subroot を commit する。

$$O_B = \text{Hash64}_{k_o}(\text{v2} \parallel R_{\text{dnsBond}} \parallel R_{\text{beacon}} \parallel R_{\text{beaconFrontier}} \parallel R_{\text{provider}} \parallel R_{\text{batch}} \parallel R_{\text{audit}} \quad (9)$$

$$\parallel R_{\text{laneDAA}} \parallel R_{\text{nullifier}} \parallel R_{\text{checkpoint}} \parallel R_{\text{params}}). \quad (10)$$

UTXO update と overlay root update は同じ atomic state transition に入れる。reorg は block-indexed delta を attach/detach する。DB read failure を zero root に置換しない。

8. Selected-Parent EVM と Native-Asset Bridge

EVM lane は block の EVM parent を GHOSTDAG selected parent に束縛する。chain block は mergeset 内の EVM payload を実行し、自身の payload は selected child によって受理される。この一段 delay により execution result は block parent の append-only function となる。virtual reorg は unbounded replay ではなく既計算 state pointer の切替になる。

header は own EVM payload と accepted mergeset execution result を別々に commit する。execution environment は chain id、fork rule、timestamp clamp、gas cap、base-fee arithmetic、skip class、state-root serialization を固定する。fixed consensus order を維持した deterministic optimistic execution の考え方を参照する [9]。

UTXO-EVM bridge は一つの native supply を保存する。deposit は EVM mint より先に UTXO value を lock し、withdraw は spendable UTXO creation より先に EVM value を burn する。

$$S_{\text{issued}} = S_{\text{UTXO spendable}} + S_{\text{bridge locked}} + S_{\text{EVM}} - S_{\text{burned}}. \quad (11)$$

EVM replay protection と fee convention は relevant Ethereum standard に従う [10, 11]。separate PQ account mechanism が activation されるまで EVM lane を PQ authorization と表現しない。

9. PALW Overview と Threat Model

PALW は deterministic LLM inference と canonical GEMM/attention schedule の audited-compute lane である。model 自体が proof を生成するという意味ではない。pre-executed work を commitment、random audit、economic exposure の後に one-shot ticket へ変換する protocol family である。

“LLM block generation” の normative meaning

LLM provider は consensus content を生成しない。parent set、transaction order、timestamp、difficulty、selected checkpoint、acceptance rule を選べない。normal node が block template を構築し、PALW 固有入力 は slot/fork binding が既に決まった matured ticket reference だけである。model output は requester へ渡され得るが、その意味内容は block validity に入らない。

9.1. Adversary capability

adversary は多数の provider identity を作り、nominal pair の両者を支配し、requester/provider/auditor/一部 DNS stake を共謀させ、cached output を再利用し、大量 leaf inventory を作り、receipt/beacon reveal を withhold し、fork を利用し、audit data を censor し、IP/ASN/funding/reward route を分散できる。また layer skip、token truncation、expert routing fixation、quantization change、nondeterministic kernel、canary-only execution により cheap path を探す。

9.2. Security assumption

hash/signature primitive が安全であること、future seed が commit deadline 前に予測不能であること、deterministic runtime profile が再現可能であること、必要な audit/dispute path の少なくとも一つが adversary に完全支配されないこと、permanent hash lane に honest work が存在することを仮定する。仮定が崩れた場合は PALW multiplier を zero に戻す。

9.3. Provider independence を仮定しない

Invariant I-8v2: No assumed independence

provider A と provider B が同一 adversary に支配される前提でも consensus safety が維持されなければならない。異なる key、bond、IP、declared operator group は independent execution の proof ではない。

replica matching が示すのは、二つの slashable key が同一 canonical receipt field に署名したことだけである。accidental fault、nondeterminism、unilateral tampering を検出できるが、work が 2 回実行されたことは示さない。従って“二つの identity が一致したから 2 GPU 分”という normalization を禁止する。

10. Deterministic LLM Work Object

10.1. Runtime profile

consensus-recognized profile は marketing model name ではなく exact manifest で識別する。最低限、次を commit する。

- model weight shard、tokenizer、architecture、quantization、sampling policy。
- runtime image、compiler/kernel graph、arithmetic width、rounding、overflow rule。
- supported accelerator class、deterministic reduction order、batching、padding、cache behavior。
- fixed shape table、operation schedule version、output serialization、reference vector。

TF32 shortcut、stochastic rounding、nondeterministic reduction、driver-selected graph change、unpinned speculative decoding は exact behavior が canonical 化されない限り禁止する。cross-vendor participation は bit-exact conformance を必要とし、満たさない場合は別 profile である。

10.2. Job precommit と work unit

requester/scheduler は job precommit を公開する。

$$J = \text{Hash64}_{k_J}(\text{policy} \parallel \text{profile} \parallel \text{shape} \parallel \text{inputCommit} \parallel \text{auditCapsuleRoot} \parallel \text{feeTerms}). \quad (12)$$

input は private でもよいが、positive credit に必要な audit policy/capsule は既に verification path を許可していなければならない。canonical job nullifier は

$$N_{\text{job}} = \text{Hash64}_{k_{N_j}}(J \parallel \text{profileId} \parallel \text{runtimeId} \parallel \text{shapeId}). \quad (13)$$

work unit は indexed token range や fixed operation bundle のような fixed quantum であり、outcome を見た後に quantum boundary を選べない。

10.3. Operation schedule と trace

各 canonical operation に identifier を与える。

$$\text{opId}_j = \text{Hash64}_{k_{op}}(\text{layer} \parallel \text{token} \parallel \text{kernel} \parallel \text{expert} \parallel \text{shape} \parallel j). \quad (14)$$

execution trace は operation order と selected intermediate projection を束縛する。

$$t_0 = \text{Hash64}_{k_t}(J \parallel \text{assignment} \parallel \text{challenge}), \quad (15)$$

$$t_j = \text{Hash64}_{k_t}(t_{j-1} \parallel \text{opId}_j \parallel \text{ioDigest}_j \parallel \text{accDigest}_j), \quad (16)$$

$$T = \text{MerkleRoot}(\{t_j\}_{j=1}^m). \quad (17)$$

trace root は commitment であって succinct proof ではない。future audit が unpredictable operation の opening を要求し、mismatch を bisect できる点に価値がある。

10.4. Output commitment と privacy

service output は salt する。

$$O = \text{Hash64}_{k_o}(N_{\text{job}} \parallel \text{outputSalt} \parallel \text{canonicalOutputBytes}). \quad (18)$$

public consensus data には O 、 T 、nullifier、shape/profile id、DA commitment を置く。raw prompt、hidden state、full output は audit policy が許す path でのみ disclose する。objective rerun も strong proof も拒否する private job は service transaction としては有効でも PALW multiplier 0 である。

10.5. Nullifier hierarchy

Version 2 は役割別 nullifier を持つ。

$$N_{\text{work}} = \text{Hash64}_{k_{Nw}}(N_{\text{job}} \parallel T \parallel O \parallel \text{quantumIndex}), \quad (19)$$

$$N_{\text{receipt}} = \text{Hash64}_{k_{Nr}}(N_{\text{work}} \parallel \text{providerBond} \parallel \text{assignmentId}), \quad (20)$$

$$N_{\text{ticket}} = \text{Hash64}_{k_{Nt}}(N_{\text{work}} \parallel \text{batchId} \parallel \text{leafIndex} \parallel \text{authorityKey}). \quad (21)$$

semantic duplicate heuristic は off-consensus monitoring に使えるが、consensus dedup は全 node が再現できる canonical commitment/nullifier に限定する。

11. Local LLM Participation と Paid Demand

PALW は、利用者が日常的な interactive local model を実行しながら、同じ deterministic execution から mining-eligible receipt を生成できるように設計する。application path と mining path は execution を共有するが、disclosure policy と credit policy は共有しない。job は完全 local、future confidential audit 許可、beacon-assigned provider への replication、第三者の paid request のいずれかとして execution 前に固定される。class は block header の自己申告ではなく committed job policy から validator が導出する。

Local-use invariant

local prompt は、その committed execution path を protocol が objective に検証できる範囲だけ PALW work になり得る。strict opaque local use は application として valid だが consensus work 0。positive private credit には future-randomized audit capsule または supported strong proof が必要であり、full Replica-class credit には beacon-assigned replica への配送が必要である。

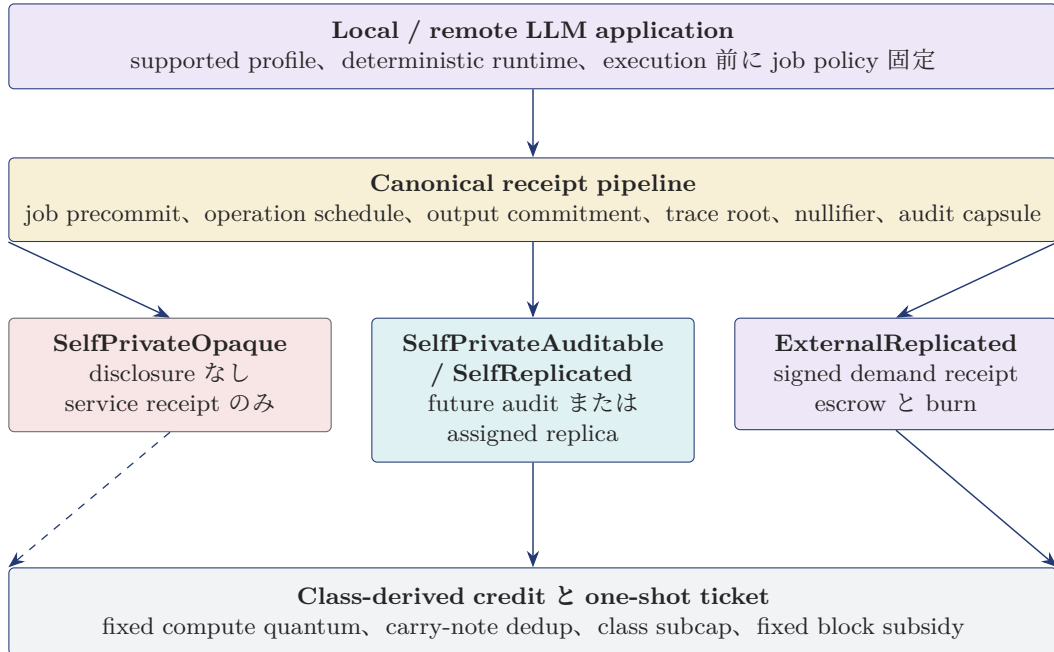


図 2. 通常の local inference と paid external inference は同じ receipt pipeline に入るが、privacy、audit、fee、credit rule は異なる。

11.1. Job class と privacy mode

job origin と assurance class は直交する。SelfReplicated と表示された job でも、ReplicaMatched に留まり objective audit が未成熟なら work 0 である。表 2 の multiplier は全 required assurance が mature した後だけ適用する。

表 2. Version 2 job class。multiplier は assurance maturity 後にも適用。

Class	Data path	Required objective evidence	Mature multiplier
SelfPrivateOpaque	prompt/output は local device 外へ出ない	commitment のみ。accepted rerun/proof path なし	0
SelfPrivateAuditable	local execution。threshold-encrypted audit capsule または supported confidential rerun	future-randomized opening、trace check、certificate。TEE 単体は不足	1/4
SelfReplicated	local user が primary。encrypted job を beacon-assigned replica へ配送	matching receipt、future audit、maturity、両 bond liability	1
ExternalReplicated	requester escrow。beacon-selected primary/replica	SelfReplicated evidence + valid demand receipt、settlement、burn	$1 + B_{\text{ext}}/C_{\text{base}}$
ProtocolCanary	protocol generated indistinguishable audit job	public canary transcript	work 0、audit payment のみ
StrongTransparent-Proof	supported proof system を持つ private/public job	dedicated verifier/version で proof verify	separate class

SelfPrivateAuditable を選ぶ利用者は、future audit draw で選ばれた場合に committed capsule を指定 audit path へ開く可能性を受け入れる。maturity 前に opening が unavailable なら job は work 0 のまま、positive-credit obligation を受け入れた後の拒否は job policy に従い slashable である。無条件の local secrecy が必要なら SelfPrivateOpaque を選ぶ。privacy を unverifiable multiplier にしないための境界である。

11.2. Interactive local workflow

local client は supported runtime の周囲に PALW mining daemon を統合する。

1. execution 前に profile、maximum shape、job class、audit policy、fee terms、salted input commitment を signed job precommit へ固定する。precommit は applicable assignment/audit seed より前に past-relative PALW registry へ accept されなければならない。registration が遅くとも利用者へ結果を返せるが mining-eligible にはならない。
2. deterministic runtime が通常の inference を実行し、canonical operation schedule、output commitment、trace root、compute counter、encrypted audit capsule を生成する。raw prompt/answer bytes は chain 外に残る。
3. SelfReplicated では DNS beacon と active provider snapshot から counterpart を導出する。local user は provider B を選べない。prompt は B へ別途暗号化する。supported confidential-execution mode がなければ、B は execution boundary 内で plaintext を扱う。
4. receipt は all-leaf batch に入り、 R_{audit} 前に freeze され、objective audit、certificate、evidence window 後にのみ mature する。
5. local node は mature compute quantum を owner-bound ticket leaf として蓄積し、derived slot/chain commitment でのみ一度消費する。

precommit requirement は、client が大量の private inventory を先に計算し、後の randomness を見て有利な job だけ公開することを防ぐ。implementation は bounded interactive slot を事前予約できるが、各 slot は fee、liability reservation、expiry、unique sequence を持ち、assignment seed 判明後に recycle できない。

11.3. Raw token ではなく canonical compute unit

prompt/output token count は telemetry には使えるが直接 work ではない。同じ token を長く出力する request も canonical decode work は消費するが、長いという理由だけで external demand にはならない。profile p の consensus cost table は prefill token、decode token、attention block、active expert projection、KV-cache update の minimum-adversarial coefficient を定義する。

$$C_{\text{base}} = \left[a_{\text{pre}}^{(p)} n_{\text{pre}} + a_{\text{dec}}^{(p)} n_{\text{dec}} + a_{\text{attn}}^{(p)} n_{\text{attn}} + a_{\text{expert}}^{(p)} n_{\text{expert}} + a_{\text{kv}}^{(p)} n_{\text{kv}} \right]_{Q_0}, \quad (22)$$

ここで $\lfloor x \rfloor_{Q_0} = Q_0 \lfloor x / Q_0 \rfloor$ 。各 count は provider の token field ではなく、frozen shape と canonical trace schedule から導出する。coefficient は既知の最安 compliant implementation に対する conservative lower bound であり、versioned hard fork でのみ変更する。receipt は service accounting 用の bucketed token metadata を公開できるが、validator は protocol object から C_{base} を再計算する。

Equation (22) は semantic value を評価しない。自己生成 job が無意味でも、実際に canonical work を行えば base work にはなり得る。protocol が無料で認めないのは、その job を *external demand* と偽る部分である。

11.4. Demand receipt, escrow, wash-trade resistance

external job は requester が provider assignment 前に unique escrow を lock し、job、profile、limit、expiry、settlement policy へ bound した demand receipt を署名した場合だけ mining-eligible である。

Listing 1. External demand receipt sketch.

```
ExternalDemandReceiptV2 {
  version, network_id, job_commitment,
  requester_ephemeral_key, requester_nonce,
  requested_model_profile_id, max_input_tokens,
  max_output_tokens, escrow_outpoint,
  fee_amount, settlement_policy_id,
  expiry_daa_score, requester_signature
}
```

別 wallet は independent demand の証明ではない。requester と provider が同一 controller でもよく、consensus は identity 問題を解いたことにしない。successful settlement では、escrow F を

$$(F_A, F_B, F_{\text{audit}}, F_{\text{burn}}) = \left(\frac{7}{20}F, \frac{7}{20}F, \frac{1}{10}F, \frac{1}{5}F \right) \quad (23)$$

へ配分する。すなわち primary 35%、replica 35%、audit/data-availability pool 10%、irreversible burn 20%である。matching delivery と required certificate state 後に settle する。provider fault 時は未獲得 provider share を requester へ返し、objective audit cost は provider liability から回収する。assignment 後の requester cancel には bounded cancellation burn を課す。

paid-demand bonus は次である。

$$\rho_E = \max \left(\rho_{\min}, 2 \text{EWMA}_8 \left(\frac{R^{\text{PALW}}}{\max(C^{\text{mature}}, C_{\min})} \right)_{<E} \right), \quad (24)$$

$$B_{\text{ext}} = \min \left(\left\lfloor \frac{C_{\text{base}}}{5} \right\rfloor, \left\lfloor \frac{F_{\text{burn}}}{\rho_E} \right\rfloor, B_{\text{remaining}}^{\text{epoch}}, B_{\text{remaining}}^{\text{provider}} \right). \quad (25)$$

ρ_E の全 input は finalized past epoch から取得し selected-parent view で解決する。bootstrap/history 不足時は $B_{\text{ext}} = 0$ 。第 1 bound は uplift を base compute の 20%以下、第 2 bound は irreversible burn が protocol の conservative bonus valuation 以上であることを要求する。 ρ_E の factor 2 により、requester と両 provider を 1 adversary が支配しても circular self-request の expected value は negative になる。残る bound は concentration と急激な demand-credit burst を制限する。

11.5. Credit schedule, class subcap, quantumization

assurance maturity 後の job credit は次である。

$$C_{\text{job}} = \begin{cases} 0, & \text{SelfPrivateOpaque,} \\ \lfloor C_{\text{base}}/4 \rfloor, & \text{SelfPrivateAuditable,} \\ C_{\text{base}}, & \text{SelfReplicated,} \\ C_{\text{base}} + B_{\text{ext}}, & \text{ExternalReplicated,} \\ 0, & \text{ProtocolCanary.} \end{cases} \quad (26)$$

Version 2 は global compute cap 内でさらに

$$C_{\text{private}} \leq \frac{H}{32}, \quad C_{\text{demand}} \leq \frac{H}{32}, \quad C_{\text{replica}} \leq r_{\text{replica}}H \quad (27)$$

を強制する。これは per-block recommendation ではなく GHOSTDAG consensus order 上の component cap である。future hard fork は直ちに lower できるが、raise には新しい activation review が必要である。

1 ticket leaf は fixed quantum Q を表す。 $c_{\text{in}} < Q$ を mature carry note とすると、

$$n_{\text{leaf}} = \left\lfloor \frac{C_{\text{job}} + c_{\text{in}}}{Q} \right\rfloor, \quad (28)$$

$$c_{\text{out}} = (C_{\text{job}} + c_{\text{in}}) \bmod Q. \quad (29)$$

carry note は non-transferable で owner、profile-family、job-class、proof-class へ bound し、one-shot nullifier、base/demand の別 component、 Q 未満、2 PALW epoch expiry を持つ。split、class 間 merge、fork reuse を禁止する。小 job の端数を利用しつつ、1 execution を arbitrary 多数の lottery leaf へ分割させない。

11.6. Fixed block reward と role allocation

token count と external bonus は coinbase 額を変えない。mature quantum 数、したがって eligible ticket 保持確率だけへ影響する。fixed subsidy R_B を持つ credited PALW source block では、Version 2 は

$$R_{\text{primary}} = \frac{7}{10} R_B, \quad R_{\text{replica}} = \frac{1}{5} R_B, \quad R_{\text{audit/DA}} = \frac{1}{10} R_B \quad (30)$$

とする。primary は self job の local provider または external job の beacon-selected primary であり、その key が ticket authority となる。replica beneficiary は certified leaf で固定する。replica のない positive-credit class では replica share を primary へ回さず burn する。audit/DA reward は objective participation record から settle する。duplicate、revoked、over-cap、zero-credit、red use は provider reward 0 である。

この配分は、requester service fee が execution/availability を補償し、fixed block subsidy が scarce winning eligibility を補償し、demand bonus は issuance も generated text validity も変更しないことを分離する。

12. Provider Assignment と Sybil-Resilient Exposure

12.1. Past-relative provider snapshot

provider candidate set は assignment seed 前に固定する。各 entry は bond outpoint、owner key、supported profile、runtime class、bond age、free slash liability、remaining capacity、probation、pending exposure、objective audit history を持つ。local latency/network observation は含めない。

12.2. Liability-weighted selection

identity count による lottery は Sybil-positive である。同じ bond を多数 ID に分割すると抽選口数が増える。Version 2 は usable liability と capacity から selection weight を導く。

$$w_i = \min \left(\left\lfloor \frac{L_i^{\text{free}}}{\rho_{\text{unit}}} \right\rfloor, Q_i^{\text{capacity}}, Q_i^{\text{epoch}}, Q_i^{\text{pair}} \right). \quad (31)$$

R_{assign} 、provider-set root、job precommit から deterministic weighted sampling without replacement を行う。同一 total liability を identity に分割しても aggregate selection probability が定義済み rounding bound を超えて増えてはならない。

12.3. Control domain

same bond outpoint、owner key、device key、prohibited role key のような consensus-visible hard link は pair を reject できる。declared operator group は conservative pair/exposure limit に使う。IP、ASN、cloud region、funding graph、reward timing、traffic fingerprint は off-consensus scheduler が correlated provider を避ける補助にできるが、validator ごとに観測が異なるため block invalid/slash rule にはしない。

12.4. Pair reuse、probation、exposure

pair は rolling exposure limit と reuse cooldown を持つ。new provider は low assignment/credit cap の probation から始める。provider i の required bond は unresolved risk に連動する。

$$B_i \geq \gamma \frac{V_i^{\text{pending}}}{p_i^{\text{detect, lower}}}. \quad (32)$$

$\gamma > 1$ は safety margin、 V_i^{pending} は immature reward と active-ticket liability、 $p_i^{\text{detect, lower}}$ は protocol lower bound である。unbond/reward release は全 evidence window 終了後とする。

13. 三段階に分離した Randomness

pair assignment、audit selection、ticket eligibility に同じ seed を使うと、registrant が job id を grind して複数段階で有利な結果を得られる。Version 2 は domain と deadline を分離する。

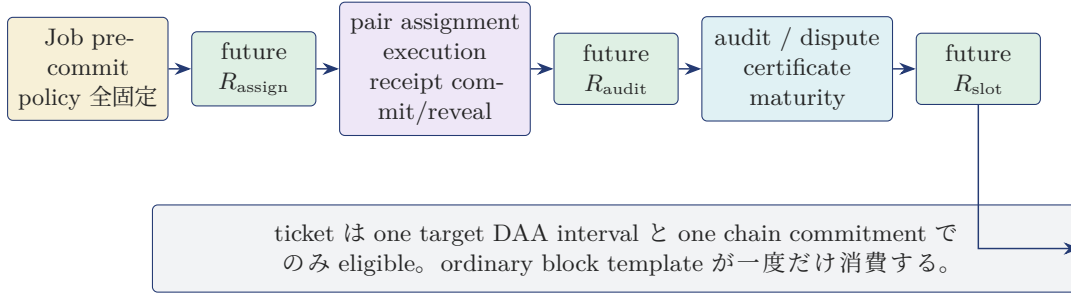


図 3. PALW timeline。各 future seed は、その seed が選択する value が commit された後にのみ確定する。

required ordering は

$$J \prec R_{\text{assign}}, \quad (\text{receipts, allLeaves, DA}) \prec R_{\text{audit}}, \quad (\text{certificate, activeSet}) \prec R_{\text{slot}}. \quad (33)$$

である。selecting seed 確定後に追加された value は signature が正しくても invalid である。

13.1. Receipt commit/reveal

provider は receipt hash を先に commit し、両 commit final 後に canonical receipt を reveal する。片方が相手の reveal を待って field をコピーすることを防ぐ。ただし同一 operator が両 commit を制御できるため 2 実行の proof ではない。二つの independently slashable signed statement を作る。

13.2. Audit 前の全 leaf 公開

batch manifest は leaf count、chunk count、profile、audit policy、bond reserve、registration deadline を固定する。全 leaf は R_{audit} より先に indexed chunk で publish/commit する。favorable な leaf だけ後出しできない。missing/duplicate chunk があれば state は advance しない。

14. Objective Audit, Dispute, Certification

14.1. Assurance class

表 3. PALW assurance class。multiplier/cap は hard-fork parameter。

Class	Required evidence	Interpretation	Initial credit
ServiceOnlyPrivate	output/trace commitment のみ	Service record。objective rerun/strong proof なし	0
ReplicaMatched	matching signed receipt 2 件	consistency/double liability、audit pending	0
AuditedPrivate	committed private-audit capsule、future challenge、passed opening/dispute	low-credit local class。selected 時は strict opacity を維持しない	1/4 origin multiplier
AuditedReplica	future-randomized audit policy、passed sample/dispute、mature certificate	no-independence assumption 下の probabilistic/economic assurance	Conservative
TEE-assisted audited	AuditedReplica + supported attestation	operational signal のみ。TEE は authority ではない	automatic uplift なし
StrongTransparent-Proof	supported proof が canonical computation を verify	future separate proof class	separate cap

miner は header で高い assurance class を選べない。class は certified leaf state と active protocol version から導出する。Section 11 の job origin と assurance class を決定論的に組み合わせ、どちらかが条件不足なら multiplier 0 とする。

14.2. Future audit draw

Audit plan は R_{audit} 、committed leaf root、provider-set root、policy の pure function である。

$$A = \text{AuditPlan}(R_{\text{audit}}, R_{\text{leaf}}, R_{\text{provider}}, \text{policy}). \quad (34)$$

leaf、trace position、rerun sentinel、DA challenge、auditor を選択する。provider は full trace/leaf inventory を commit する前に selected position を知れない。

14.3. Canary と real-job audit

protocol canary は ordinary job と同じ envelope、padding、timing window、routing path で配送する。runtime を全く実行しない provider の検出には有効だが、canary classifier を学習され得るため canary-only は不十分である。positive real-job credit は requester-authorized rerun、threshold-escrow opening、supported confidential rerun、strong transparent proof のいずれかを許可しなければならない。どれも許可しない job は service-only である。

14.4. Mismatch resolution

第三 provider を追加して多数決にしない。A/B が同一 operator なら $A = B \neq C$ でも正しさは決まらない。mismatch は leaf/batch を Quarantined に移し、reference rerun、trace bisection、strong proof で解決する。高価な計算を disputed step まで絞る考え方は interactive verification に関連する [12, 13]。

matrix multiplication $AB = C$ の dispute では random field vector r を選び

$$A(Br) = Cr \quad (35)$$

を検査する Freivalds-style check が多くの false claim を低コストで reject できる [14]。これは complete LLM proof ではなく dispute accelerator である。

14.5. Certificate semantics

DNS-auditor certificate は objective fact に署名し、provider independence や generated text truth を宣言しない。V2 certificate は次を commit する。

- provider-set / eligible-auditor snapshot root。
- job、pair assignment、receipt commit/reveal、leaf、DA root。
- audit policy、selected plan、response、rerun/dispute result、active-leaf bitmap。
- reserved liability / exposure root。
- certificate version、network、fork、epoch、maturity interval。

vote は auditor identity で sort/dedup し、signature と fixed eligible snapshot weight を検査する。eligible stake 0 の zero-over-zero quorum は成立しない。conflicting vote は objective evidence である。

15. Ticket Creation と LLM-Work-Backed Block Generation

15.1. Batch state machine

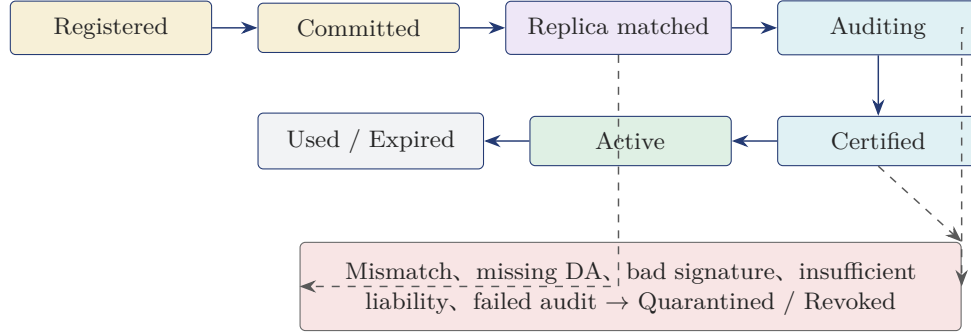


図 4. PALW leaf/batch lifecycle。Active、mature、non-revoked leaf だけが block ticket になれる。

15.2. Target slot

active set freeze 後、 R_{slot} が各 leaf に one target DAA interval を割り当てる。

$$S = \text{Slot}(R_{\text{slot}}, \text{batchId}, \text{leafIndex}, R_{\text{leaf}}, \text{slotWindow}). \quad (36)$$

header の S は derived result との比較用であり miner-selectable ではない。window endpoint は checked arithmetic で処理する。

15.3. Checkpoint chain commitment

P_S を $S - L$ 以下の latest DNS-finalized checkpoint、 L を hard-fork lookback とする。

$$K_S = \text{Hash64}_{k_{\text{chain}}}(\text{network} \parallel \text{fork} \parallel P_S \parallel \text{dnsCertCommit} \parallel S). \quad (37)$$

checkpoint が存在しなければその slot の PALW lane は Halted である。zero placeholder を checkpoint として扱わない。

15.4. Expected lane difficulty と eligibility

selected-parent lane-DAA state が $\text{bits}_S^{(C)}$ を決める。ticket eligibility digest は

$$z = \text{Hash64}_{k_e}(R_{\text{slot}} \parallel K_S \parallel S \parallel N_{\text{ticket}} \parallel \text{proofClass} \parallel \text{paramsHash}) \quad (38)$$

であり、 $\text{Uint512}(z) \leq \text{target}_{512}(\text{bits}_S^{(C)})$ のとき eligible である。canonical nonce は N_{ticket} の fixed 64-bit projection とし、second grinding variable にしない。

15.5. Header V3 field

表 4. Header-v3 PALW/component-work field。

Field	Meaning / validation
blue_hash_work	cumulative credited blue hash-lane work。GHOSTDAG から再計算。
blue_compute_work	effective-work formula 前の cumulative compute work。consensus が再計算/cap。
palw_batch_id	referenced leaf を含む Active batch。
palw_leaf_index	committed leaf root 内の canonical index。
palw_ticket_nullifier	first-class one-shot nullifier。nonce/authorization を束縛。
palw_epoch_certificate_hash	active V2 certificate と maturity state の hash。
palw_chain_commit	Equation (37) の lagged checkpoint binding。
palw_target_daa_interval	Equation (36) の derived slot。
palw_authorization_hash	body-carried ML-DSA block authorization の hash。
palw_proof_type	derived assurance/proof-class discriminant。

15.6. Template algorithm

block producer は block assembly 中に LLM を実行しない。active ticket を選び、validator と同じ pure resolver を実行する。

Listing 2. Conceptual PALW template construction。

```

view = resolve_selected_parent_view(parents)
for ticket in view.active_ticket_inventory:
    if ticket.target_interval != next_daa_interval: continue
    binding = resolve_ticket_binding(view, ticket)
    if binding.expected_bits != template_bits: continue
    if binding.delta_compute > view.compute_headroom: continue
    if not one_shot_eligibility(binding): continue
    auth = sign_block_authorization(ticket.authority_key, header_preimage)
    return build_header_v3(view, binding, auth)
return build_hash_lane_header(view)

```

local scheduler は validity を変更できない。equally valid candidate の選択は自由だが、各 candidate は独立に objective predicate を満たす必要がある。

15.7. Authorization

ticket authority は exact block preimage、network、fork、ticket nullifier、slot、chain commitment、certificate、expiry に domain-separated signature を作る。different header への再署名は objective double-authorization evidence である。authorization object は body に置き、その hash を header が commit する。

16. Fast Consensus Verification

validator は selected parent から immutable PALW view を一つ解決し、block を検証する。missing state、unknown proof class、component work inconsistency、malformed signature、arithmetic overflow、required witness unavailable は rule error である。

16.1. Canonical validation clause

algo-4 block は次を atomic に全て満たす場合だけ valid である。

1. header version/algo id が untrusted self-declared score ではなく actual selected-parent DAA state で active。
2. header shape が canonical。ticket-specific field は必要箇所 nonzero、禁止箇所 zero。
3. batch、manifest、leaf、receipt、certificate、maturity、audit、revocation state が selected-parent snapshot に存在。
4. profile、runtime、shape、proof class、provider bond、authority key が supported かつ certificate と一致。

5. signature、vote uniqueness、stake weight、phase deadline、certificate root を検証。
6. ticket/work nullifier が canonical かつ selected-parent past で未使用。
7. target slot S を committed active set と R_{slot} から導出。
8. lagged finalized checkpoint と K_S を導出し header と一致。
9. lane DAA が exact expected bits を導出。
10. one-shot eligibility digest が target を満たし nonce が canonical。
11. body authorization が header hash field と一致し ticket authority で verify。
12. job class を precommit/certificate から導出。fully opaque、unmatched、unaudited positive-credit claim は active stage に従い reject または zero credit。
13. external job では demand receipt、unique escrow outpoint、settlement state、fee burn、expiry、requester signature を検証し、finalized past data から B_{ext} を再計算。
14. base compute、origin/assurance multiplier、carry-note transition、quantum count、class subcap、reward output を完全再現。
15. compute increment を proof class、job class、profile、shape、rollout stage、minimum adversarial cost から導出。
16. full increment が deterministic compute headroom に収まる。partial credit は不可。
17. recomputed blue_hash_work、blue_compute_work、blue_work、blue_score が header/GHOSTDAG result と一致。
18. overlay post-state、active-nullifier update、reward class、pruning witness が同じ accepted result を commit。

これらは同時 activation する。header check だけ先に有効化し、chain commit、lane bits、cap、authorization を defer する intermediate protocol は許可しない。

16.2. GHOSTDAG ticket dedup

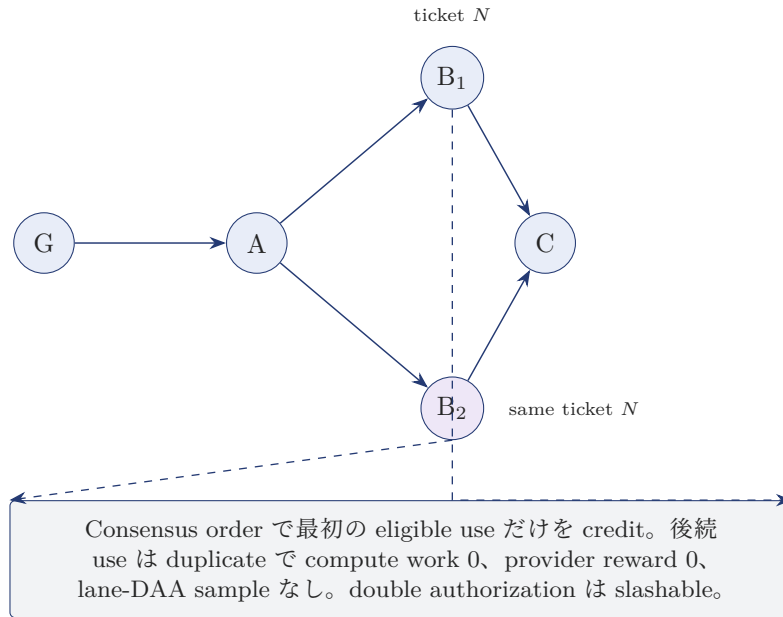


図 5. Sibling fork での ticket reuse は local first-seen time ではなく deterministic GHOSTDAG order で解決する。

active-nullifier view は block-indexed/reorg-safe である。retention は ticket expiry、private-fork replay、evidence window、maximum supported reorg、pruning reconstruction を cover する。ticket の価値期間より短い cache は consensus dedup ではない。

16.3. Pruning と IBD

pruning bundle は finalized DNS certificate commitment、provider/eligible-set root、active batch/leaf registry、audit result root、lane-DAA state、active-nullifier root、component-work state、rollout parameter、first post-pruning block 検証に必要な Merkle witness を含む。pruned node と archival node は同じ PALW predicate を導出しなければならない。header component field は比較対象であり trusted state ではない。

17. Work Accounting, Compute Cap, Lane DAA

17.1. Minimum adversarial cost

Replica credit は honest provider の claimed FLOPs、GPU count、電気代、cloud invoice ではなく、両 provider identity を支配する adversary が利用できる cheapest plausible canonical execution を基準にする。hard-forked profile/shape table が work unit を ΔC_{raw} に map する。provider 自身が weight を申告しない。

source block b の increment は

$$\Delta C_b = \mu_{\text{rollout}} (\mu_{\text{assurance}} \mu_{\text{origin}} C_{\text{base},b} + B_{\text{demand},b}). \quad (39)$$

$C_{\text{base},b}$ は Equation (22) から導出し、multiplier は exact rounding を持つ rational consensus parameter である。demand term は Equation (25) を満たす場合だけ nonzero。source block の stage は acceptance 時に固定し、後の parameter change で historic work を再評価しない。

17.2. Component recurrence

selected-parent/mergeset consensus order 後の component work を (H_p, C_p) とする。hash source は ΔH_b を加え、eligible PALW source は full increment が収まる場合だけ ΔC_b を加える。

$$H' = H_p + \sum_{b \in M_H} \Delta H_b, \quad (40)$$

$$C' = C_p + \sum_{b \in M_C}^{\text{consensus order}} \mathbf{1}[\Delta C_b \leq \text{headroom}_b] \Delta C_b, \quad (41)$$

$$\text{headroom}_b = \min \left(r_{\text{replica}} H_b - C_{\text{replica},b}, \frac{H_b}{32} - C_{\text{private},b}, \frac{H_b}{32} - C_{\text{demand},b}, 4H_b - C_{\text{total},b} \right). \quad (42)$$

source class に関係する cap だけを minimum へ含める。over-cap block に partial credit を与えない。partial credit は reward、DAA sample、security accounting を fractional remnant に依存させる。fork rule に従い valid-shadow または red とし、compute increment/provider base reward は zero にする。

17.3. Effective work と attack bound

Equation (1) が effective blue work を与える。Replica class の staged cap は adversarial testnet で

$$r_{\text{replica}} \in \left\{ 0, \frac{1}{16}, \frac{1}{8}, \frac{1}{4} \right\} \quad (43)$$

を提案する。1/2 は Version 2 design ceiling であって launch target ではない。全 Replica ticket が forged でも permanent hash component は残る。ただし exact takeover threshold は work distribution と DNS gate に依存するため単一の安心用 percentage に縮約しない。

17.4. Lane-specific DAA

Hash/PALW block は別 sample window/target rate を持つ。PALW DAA は unique、active、blue、nonzero-credit source block だけを sample する。duplicate、revoked、red、over-cap、zero-rollout block は PALW target を動かさない。sparse window は previous bits または specified bootstrap target を保持し、timestamp clamp/max adjustment が stockpiled ticket の burst release を抑える。

slot rule と lane DAA は同じ selected-parent view から解決する。candidate block が自分を判定する difficulty を変更してはならない。

18. Reward, Slashing, Liveness

18.1. Reward escrow と settlement

provider compensation は ticket expiry、fraud-evidence window、maximum supported reorg horizon の全てより後まで lock する。Version 2 は Equation (30) の role allocation、すなわち primary 70%、replica 20%、audit/DA 10%を使用する。total subsidy は monetary schedule で固定し token count/job class に依存しない。certified class に replica がなければ replica share を burn する。

External service fee は Section 11 の別 settlement 35/35/10/20 を使う。provider share と PALW subsidy share は unresolved liability に対して escrow する。audit/DA claim は objective audit、retrieval、dispute record からのみ支払う。後に objective fraud evidence が成立した job/block は associated bond と未成熟 role reward を slash できる。

18.2. Fraud expected return

reward R 、detection lower bound p 、slashable bond/locked reward $B + L$ 、fee F 、attack compute cost K に対し conservative model は

$$\mathbb{E}[\Pi_{\text{fraud}}] \leq (1 - p)R - p(B + L) - F - K. \quad (44)$$

である。activation は honest two-GPU assumption ではなく plausible low-cost attack の下で右辺が負となる parameter regime を要求する。

18.3. Objective evidence

slashable evidence は conflicting signed receipt commit/reveal、double ticket authorization、invalid audit opening、provably inconsistent trace branch、duplicate auditor vote、false certificate root、required DA response missing、on-chain key に束縛された role violation を含む。IP similarity、region、funding heuristic、model-quality disagreement は slash evidence ではない。

18.4. Historic work

finality 後に fraud が見つかったとしても historic GHOSTDAG work を遡及削除しない。future discovery に過去 validity を依存させるためである。代わりに unused ticket revoke、bond/locked reward slash、provider/auditor quarantine、future multiplier 0、evidence root commit を行う。

18.5. Degraded/Halted mode

DNS beacon quorum 不足、finalized checkpoint unavailable、audit DA missing、selected-parent overlay unreplicable の場合、PALW は deterministic mode に入る。

- **Healthy**: new batch activation 可、configured multiplier 適用。
- **DegradedGrace**: new activation 不可、existing ticket は zero または明示的 reduced credit。
- **Halted**: algo-4 block reject、hash-lane production は継続。

hash lane は security floor と同時に liveness fallback である。

19. Security Analysis

表 5. 主要 attack、control、residual risk。

Attack	Primary control	Residual risk
A/B Sybil collusion	no-independence assumption、one-execution cost floor、liability-weighted selection、pair/exposure cap、future audit	physical common control の完全識別は不能
Fabricated output/-trace	receipt commit/reveal、all-leaf publication、random opening、rerun/dispute、slash/escrow	unsampled fraud は Replica class で probabilistic
Partial model execution	frozen schedule、assignment challenge、trace opening、reference rerun	unknown cheaper canonical-equivalent path は cost floor を下げる
Canary-only honesty	indistinguishable delivery、real-job audit capability、future selection	perfect canary indistinguishability は仮定しない
Leaf grinding	precommit deadline、per-leaf liability/fee、inventory quota、future audit/slot seed	high-capital actor は legitimate attempt を増やせる
Same work in many leaves	job/work nullifier、quantum index、batch dedup、exposure accounting	semantic duplicate は objective detection が難しい
Ticket double/fork reuse	header nullifier、slot/chain commit、authorization、GHOSTDAG dedup	private fork は slash evidence を作るが multiple credit は不可
Malicious DNS cert	objective certificate field、public vote、slashing、hash floor、Replica subcap	threshold DNS compromise は compute lane を損傷可能
Audit collusion	future committee、public transcript、reference dispute、role separation	allowed verifier 全支配時は Replica soundness 失敗
Privacy leakage	salted output、audit capsule、selective disclosure、service-only zero credit	positive credit は auditability と privacy の trade-off
Runtime differential	exact manifest、class matching、golden vector、fail-closed activation	hardware ecosystem が profile 分裂する可能性
Self-query inflation	actual canonical compute、fixed quantum、per-provider exposure、class cap	meaningless でも real な compute は base work になり得る
External-demand wash trading	unique escrow、20% burn、finalized reward/CU history による bonus pricing、20% base uplift ceiling	market-value estimate は急変に遅れる可能性
Carry-note replay/-class laundering	owner/class/profile binding、one-shot nullifier、2 epoch expiry、base/bonus component 分離	selected-parent state/pruning witness が増える

19.1. Provider collusion

consensus に metadata から independent human を認識させないことが中心変更である。A/B agreement は two liabilities と accidental-error reduction を与えるが、work credit は one adversarially shared execution を前提にする。future resource-bound proof が physical replication を示せるなら stronger proof class と別 accounting を持つべきである。Filecoin の proof-of-replication は identity count ではなく resource-bound proof が必要であることの対照例である [15]。

19.2. Fake/shortened computation

provider は layer skip、token truncation、expert routing fixation、KV reuse、low precision、smaller model substitution を試みる。profile binding、schedule commitment、random trace opening、reference rerun が deviation を検出する。ただし trace root 単体は proof ではなく、same canonical output/opening をより安く生成する unknown shortcut が見つければ cost table を下方修正する。consensus は effort の善意ではなく minimum known adversarial cost を credit する。

19.3. Ticket inventory/grinding

全 leaf は audit randomness 前、全 active ticket は slot randomness 前に固定する。per-leaf bond reserve、fee、capacity quota、expiry により inventory を有料化する。legitimate work を多く支払って draw を増やすことと、hidden free draw を後出しすることは区別する。

19.4. DNS/auditor compromise

DNS は semantic correctness/provider independence を certify せず deterministic transcript を certify する。それでも threshold coalition は validator が underlying fact を検証しない、または opaque fact を許す場合に false certificate を作れる。Version 2 は certificate field を publicly reproducible root に限定し、full node が state transition を検証する。hash work/class cap が amplification を bound し、slashing/public vote が accountability を与える。total DNS compromise は explicit residual risk である。

19.5. Self-generated workload と token inflation

PALW は local prompt の社会的 usefulness を判定しない。attacker は repetitive text を生成させられるが、credit は requested token number ではなく canonical operation schedule と minimum adversarial cost から導出する。したがって attack は実際の bounded compute を消費し、provider exposure、ticket expiry、lane DAA、Replica cap、private-work subcap に従う。protocol が区別するのは、この work を external demand と偽る行為である。

19.6. Circular external demand

miner は requester wallet を作り自分へ発注できる。identity heuristic では確実に防げないため、Version 2 は unique escrow と irreversible burn を要求し、Equation (25) で bonus CU を finalized historical reward rate の少なくとも 2 倍で評価する。1 controller が requester と両 provider share を回収しても burn を失い、bonus 単体の expected value は positive にならない。base compute は canonical computation が実行・audit された範囲だけ残る。

19.7. Private-local downgrade attack

client は opaque job を後から audited と label 変更したり、provisional credit 取得後に selected opening を拒否したりできない。job class/audit policy は execution 前に commit し、provisional receipt は GHOSTDAG work 0、matured certificate だけが multiplier を決める。missing opening を empty transcript へ normalize しない。job を service-only に留めるか、positive-credit obligation 受諾後なら objective slash evidence とする。

19.8. Prompt injection と generated content

model は node API を呼べず、block field を選べず、ticket authority message を署名できないため、prompt injection は consensus を変更できない。generated bytes は committed output data として扱う。commercial service が agentic tool を提供しても、その permission は mining permission ではない。

19.9. Denial of service

PALW object は size、count、recursion、signature、allocation bound を持つ。parser は allocation 前に length を検査する。unknown proof type は reject する。expensive signature/Merkle check の前に cheap shape/activation check を行う。invalid header は panic せず structured rule error を返す。overlay write は atomic であり、failure を empty state に normalize しない。

19.10. Privacy と auditability

arbitrary private LLM inference を data disclosure、trusted confidential environment、requester involvement、strong proof のいずれもなしに objective verification する一般的方法はない。Version 2 はこの限界を“certificate”で隠さない。job owner は service-only privacy、threshold/opening audit、confidential rerun、future strong proof を選び、consensus credit は assurance class に従う。

20. Rollout, Test Gate, Implementation Discipline

20.1. Activation stage

表 6. PALW rollout stage。

Stage	Behavior	Replica cap
Disabled	algo-4 reject、PALW state transition なし	0
ShadowSidecar	DAG 外で work object を検証し runtime/DA/audit/attack を測定	0
LocalReceiptOnly	local application が Self/External receipt、demand escrow、carry note を生成。GHOSTDAG work なし	0
ConsensusVisibleZero- Credit	strict quota 下で algo-4 observe、work/reward/DAA sample なし	0
Adversarial-1	dedicated testnet で full validation/rollback	$H/16$
Adversarial-2	external red-team、stockpile、censorship、 all-compute-forged drill	$H/8$
Candidate	sustained gate と independent review 後のみ	initial 最大 $H/4$
Version-2 ceiling	Replica class の design ceiling、target ではない	$H/2$

$\Delta C = 0$ でも algo-4 block が blue score、anticone、DAA score、storage、epoch timing に影響するなら true weight-zero ではない。最初は DAG 外 sidecar が安全である。consensus-visible stage では残る全 effect を明示する。

20.2. No-go condition

次のいずれかが unresolved の間は nonzero Replica credit を禁止する。

- algo-4 が panic する、または complete ticket validity 前に GHOSTDAG credit を得る。
- overlay/beacon/batch/nullifier/lane-DAA state が global で block-indexed/reorg-safe でない。
- DNS stake が unit placeholder、reveal secret が seed に入らない、quorum denominator が committed stake。
- slot、chain commit、expected bits、authorization、eligibility、full-increment cap が atomic predicate に接続されない。
- template、P2P、gRPC、pruning、trusted data、IBD が全 v3 field/state witness を round-trip しない。
- positive-credit job が全 real-job audit path を拒否できる。
- local job class、canonical compute unit、demand escrow/burn、carry-note transition、coinbase role output を full node が独立再計算しない。
- self-request wash trading で demand bonus の expected value が positive になる、または finalized-history/bootstrap safeguard がない。
- reward escrow/liability reservation が enforcement されない。
- adversarial test で deterministic zero-credit rollback が実証されない。

20.3. Mandatory test

header version/algo fuzz、forged self-declared DAA score を含む activation boundary、commit/reveal/chunk/vote arrival permutation、deep reorg/crash recovery、sibling collective cap overflow、ancestor/fork nullifier reuse、sparse/burst lane DAA、reveal censorship、certificate equivocation、pruning import、cross-hardware golden vector、parser allocation limit、template-versus-validator equivalence、interactive local-client precommit race、opaque-to-audited class downgrade、demand-receipt replay、requester/provider common-control 下の wash-trade simulation、fee-split/burn arithmetic、reorg を跨ぐ carry-note double-spend を gate にする。

core property は次である。

$$\text{same selected-parent view} \Rightarrow \text{same seed, slot, chain commit, bits, cap, verdict}, \quad (45)$$

$$\text{same valid message set in any arrival order} \Rightarrow \text{same committed overlay state}, \quad (46)$$

$$\text{accepted block} + \text{crash} + \text{restart} \Rightarrow \text{same UTXO and overlay roots}. \quad (47)$$

21. 関連研究

MISAKA の DAG consensus は GHOSTDAG を継承する [1]。PQ primitive は NIST ML-DSA、ML-KEM、hash standard に従い [3, 4, 16]、BLAKE2 domain-separated commitment を用いる [5]。provider identity 分析は unconstrained distributed identity の Sybil multiplication 問題に基づく [2]。

PALW dispute は optimistic verification/interactive computation に関連し [12, 13]、Freivalds check は matrix-product dispute の subroutine になる [14]。distributed randomness research は beacon と bias-resistance limit に関係する [7, 8]。Filecoin の resource-bound replication は重要な対照である。PALW Version 2 は physical replication proof を主張しないため Replica class を conservative/separate に accounting する [15]。succinct verifiable inference/ZKML は future StrongProof class の候補であり、current audited-replica design と混同しない。

22. 結論

MISAKA Version 2 は post-quantum UTXO consensus surface、GHOSTDAG ordering、consensus-native EVM、DNS stake overlay を維持し、language model を consensus authority にせず LLM work を追加する。deterministic inference は block generation 前に実行される。日常の local model 利用は self-private または self-replicated receipt を生成でき、paid external request は狭く bound された burn-backed demand bonus を追加できる。provider は receipt/trace を commit し、全 leaf は future audit draw 前に固定され、objective audit/dispute が bounded work ticket を mature し、normal block template が derived slot/fork commitment で一度だけ消費する。full node は protocol history、job class、fee burn、compute normalization、ticket ownership、work cap を fast/deterministic に検証する。

中心的な security choice は、independent operator を識別できると装わないことである。common control を前提に minimum adversarial execution cost 以下しか credit せず、positive work を audit-capable に限定し、liability reserve、reward escrow、concentration limit、permanent hash floor、strict Replica subcap を維持する。Replica PALW は probabilistic/economic assurance であり、2 回の physical execution の mathematical proof ではない。complete state、audit、pruning、adversarial gate が満たされるまで正しい consensus weight は zero である。

A. Consensus Object Sketch

以下は explanatory sketch である。wire encoding は exact width、canonical serialization、version domain、bounded vector を必要とする。

Listing 3. Selected PALW V2 object.

```
PalwJobPrecommitV2 {
  version, network_id, job_commitment, job_nullifier,
  job_class, model_profile_id, runtime_class_id,
  shape_id, quantum_count, audit_policy,
  audit_capsule_root, registration_deadline
}

PalwComputeReceiptV2 {
  job_commitment, job_class, model_profile_id,
  runtime_class_id, shape_id, canonical_compute_units,
  prompt_tokens_bucket, output_tokens_bucket,
  output_commitment, trace_root, operation_schedule_root,
  receipt_nullifier, provider_bond, signature
}

ExternalDemandReceiptV2 {
  job_commitment, requester_ephemeral_key, requester_nonce,
  requested_model_profile_id, max_input_tokens,
  max_output_tokens, escrow_outpoint, fee_amount,
  settlement_policy_id, expiry_daa_score, requester_signature
}

PalwCarryNoteV2 {
  owner_bond, profile_family, job_class, proof_class,
  base_units, demand_units, previous_note_nullifier,
  note_nullifier, created_epoch, expiry_epoch
}
```

```

PalwProviderSnapshotEntryV2 {
    bond_outpoint, owner_key_hash, runtime_class_id, status,
    bonded_amount, reserved_liability, free_liability,
    capacity_remaining, epoch_credit_remaining, pair_credit_remaining,
    bond_age, objective_audit_successes, objective_audit_failures
}

PalwReplicaReceiptV2 {
    assignment_id, provider_bond, job_nullifier, work_nullifier,
    output_commitment, trace_root, operation_schedule_root,
    receipt_da_root, execution_challenge_root, signature
}

PalwBatchCertificateV2 {
    batch_id, manifest_root, leaf_root, active_leaf_bitmap_root,
    provider_snapshot_root, pair_assignment_root,
    receipt_commit_root, receipt_reveal_root, audit_plan_root,
    audit_result_root, dispute_result_root, exposure_root,
    eligible_auditor_root, votes, maturity_interval
}

```

B. Objective Verifier Pseudocode

Listing 4. Conceptual block verifier.

```

verify_palw_block(block, selected_parent):
    view = resolve_palw_view(selected_parent)
    require view.rollout_stage.allows_algo4
    require canonical_header_shape(block.header)

    binding = lookup_active_leaf_and_certificate(view, block.header)
    verify_manifest_leaf_receipts_certificate(binding, view)
    verify_audit_and_maturity(binding, view)
    verify_provider_liability_and_proof_class(binding, view)

    job_class = derive_job_class(binding.precommit, binding.certificate)
    base_cu = derive_canonical_compute_units(binding.profile,
                                             binding.schedule)

    if job_class == ExternalReplicated:
        verify_demand_receipt_escrow_settlement_and_burn(binding, view)
        demand_cu = derive_demand_bonus(binding, view.finalized_history)
    else:
        demand_cu = 0
    delta_c = derive_class_credit(job_class, base_cu, demand_cu,
                                  binding.assurance, view.params)
    verify_carry_note_transition(binding, delta_c, view)
    verify_reward_output_vector(block.coinbase, binding, view.params)

    require derive_slot(view, binding) == header.target_interval
    require derive_chain_commit(view, binding) == header.chain_commit
    require derive_lane_bits(view, binding) == header.bits
    require canonical_nonce(binding.ticket_nullifier) == header.nonce
    require one_shot_hash(binding, view) <= target(header.bits)
    require verify_block_authorization(block, binding)
    require not view.active_nullifiers.contains(binding.ticket_nullifier)

    require delta_c <= derive_full_headroom(view, block.mergeset)

    result = ghostdag_with_component_work(block, view, delta_c)
    require result.component_fields == block.header.component_fields
    require result.overlay_post_root == block.header.overlay_commitment_root

```

```
commit_atomically(result)
```

C. Parameter Classification

表 7. Normative/provisional PALW parameter.

Parameter	Classification	Version 2 rule
Hash floor	Normative	Permanent / nonzero
Global compute cap	Normative	Compute contribution $\leq 4H$
Replica class cap	Normative mechanism	staged、initial candidate $\leq H/4$ 、V2 ceiling $H/2$
Provider pair $k = 2$	ReplicaExact matching の normative rule	consistency only、two-execution credit 禁止
Audit probability	Hard-fork parameter	bond economics に使う conservative lower bound 必須
Local job multiplier	Normative V2 initial schedule	Opaque 0、private-auditable 1/4、self-replicated 1
External demand uplift	Normative mechanism	base の最大 1/5、burn-priced、 $H/32$ subcap
External fee settlement	Normative V2 initial schedule	primary 35%、replica 35%、audit/DA 10%、burn 20%
PALW block subsidy routing	Normative V2 initial schedule	primary 70%、replica 20%、audit/DA 10%。replica 不在 share は burn
Carry note	Normative mechanism	non-transferable、class-bound、one-shot、1 quantum 未満、2 epoch expiry
Reward split	PALW V2 では superseded	Version 1 の 77/8/15 proposal は本 revision で使用しない
Model/profile label	Non-consensus convenience	exact manifest hash が consensus identity
Target BPS split	Provisional	lane-DAA/adversarial measurement 後に activation
TEE use	Optional	authority なし、automatic credit uplift なし
Strong proof system	Research	specified 後に separate class/cap

参考文献

- [1] Y. Sompolinsky, S. Wyborski, and A. Zohar. “PHANTOM GHOSTDAG: A Scalable Generalization of Nakamoto Consensus”. In: *Cryptology ePrint Archive* 2018/104 (2018). URL: <https://eprint.iacr.org/2018/104>.
- [2] J. R. Douceur. “The Sybil Attack”. In: *International Workshop on Peer-to-Peer Systems*. 2002, pp. 251–260. DOI: [10.1007/3-540-45748-8_24](https://doi.org/10.1007/3-540-45748-8_24).
- [3] National Institute of Standards and Technology. *FIPS 204: Module-Lattice-Based Digital Signature Standard*. Tech. rep. NIST, 2024. DOI: [10.6028/NIST.FIPS.204](https://doi.org/10.6028/NIST.FIPS.204).
- [4] National Institute of Standards and Technology. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Tech. rep. NIST, 2024. DOI: [10.6028/NIST.FIPS.203](https://doi.org/10.6028/NIST.FIPS.203).
- [5] M.-J. O. Saarinen and J.-P. Aumasson. *The BLAKE2 Cryptographic Hash and Message Authentication Code*. Tech. rep. RFC 7693. IETF, 2015. DOI: [10.17487/RFC7693](https://doi.org/10.17487/RFC7693).
- [6] K. Lewi et al. “Securing Update Propagation with Homomorphic Hashing”. In: *Cryptology ePrint Archive* 2019/227 (2019). URL: <https://eprint.iacr.org/2019/227>.
- [7] E. Syta, P. Jovanovic, E. Kokoris-Kogias, et al. “Scalable Bias-Resistant Distributed Randomness”. In: *IEEE Symposium on Security and Privacy*. 2017, pp. 444–460. DOI: [10.1109/SP.2017.45](https://doi.org/10.1109/SP.2017.45).

- [8] League of Entropy. *drand: Distributed Randomness Beacon*. 2020. URL: <https://drand.love/docs/overview/>.
- [9] R. Gelashvili, A. Spiegelman, Z. Xiang, et al. “Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing”. In: *arXiv preprint arXiv:2203.06871* (2022). arXiv: 2203.06871.
- [10] V. Buterin. *EIP-155: Simple Replay Attack Protection*. 2016. URL: <https://eips.ethereum.org/EIPS/eip-155>.
- [11] V. Buterin et al. *EIP-1559: Fee Market Change for ETH 1.0 Chain*. 2019. URL: <https://eips.ethereum.org/EIPS/eip-1559>.
- [12] J. Teutsch and C. Reitwiessner. “A Scalable Verification Solution for Blockchains”. In: *arXiv preprint arXiv:1908.04756* (2019). arXiv: 1908.04756.
- [13] H. Kalodner, S. Goldfeder, X. Chen, et al. “Arbitrum: Scalable, Private Smart Contracts”. In: *USENIX Security Symposium*. 2018, pp. 1353–1370.
- [14] R. Freivalds. “Probabilistic Machines Can Use Less Running Time”. In: *IFIP Congress* (1979), pp. 839–842.
- [15] Protocol Labs. *Filecoin: A Decentralized Storage Network*. 2017. URL: <https://filecoin.io/filecoin.pdf>.
- [16] National Institute of Standards and Technology. *FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. Tech. rep. NIST, 2015. DOI: 10.6028/NIST.FIPS.202.